INVERTIS UNIVERSITY, BAREILLY
B.Tech CSE - Third Year

DBMS & SQL Lab (BCS552)

Lab Practical Record: Experiments 1 to 15

Name	Nishant
Roll No.	2410303310
Course	B.Tech CSE, Third Year
Subject	DBMS & SQL Lab (BCS552)

Index of Experiments

Experiment 1: Creating a Table and Adding Rows 3

Experiment 2: Updating and Deleting Rows 5

Experiment 3: Selecting Columns and Rows with WHERE 8

Experiment 4: Combining Conditions 11

Experiment 5: Pattern Matching with LIKE 14

Experiment 6: Sorting Rows and Removing Repeats 17

Experiment 7: Using Aggregate Functions 20

Experiment 8: Grouping Rows and Using HAVING 23

Experiment 9: Aliases and CASE 26

Experiment 10: Joining Two Tables with INNER JOIN 29

Experiment 11: Other Kinds of Joins 32

Experiment 12: Using a Subquery 35

Experiment 13: UNION and INTERSECT 38

Experiment 14: Creating and Using a View 41

Experiment 15: Indexes and Adding a Row 44

EXPERIMENT NO. 1

1 Experiment Title

Creating a Table and Adding Rows

2 Aim

To create a table, add three rows with one INSERT statement, and show the table.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

A relational database stores data in tables made of rows (records) and columns (fields). The **CREATE TABLE** statement (DDL) gives the table a name and defines each column with a data type. **INT** stores whole numbers and **VARCHAR(n)** stores text of up to n characters.

The **INSERT INTO ... VALUES** statement (DML) adds rows to a table. Several rows can be added in one statement by separating the value lists with commas. **SELECT *** displays every column of every row.

5 Algorithm / Procedure

1. Open MySQL Workbench (or the mysql client), connect to the server and select a database.
2. Write a CREATE TABLE statement for student with the columns roll_no, name and marks.
3. Write one INSERT statement that contains the three value lists.
4. Run SELECT * FROM student to display the table.
5. Compare the result with the sample output.

6 Program / Queries / Commands

Task 1: Create the student table SQL

```sql
CREATE TABLE student (
    roll_no  INT,
    name     VARCHAR(30),
    marks    INT
);
```

Task 2: Add three rows with one INSERT SQL

```sql
INSERT INTO student (roll_no, name, marks)
VALUES (101, 'Asha', 78),
       (102, 'Rohan', 64),
       (103, 'Neha', 88);
```

Task 3: Show all columns and all rows SQL

```sql
SELECT * FROM student;
```

7 Input

Table structure and test data used in this experiment.

Table student: structure

Column	Data type	Constraint
roll_no	INT	-
name	VARCHAR(30)	-
marks	INT	-

Table student: data

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88

8 Output

Output of Task 1: Create the student table

```
Query OK, table created
```

Output of Task 2: Add three rows with one INSERT

```
Query OK, 3 rows affected
```

Output of Task 3: Show all columns and all rows

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88

```
3 rows in set
```

9 Result

The student table was created and three rows were added with a single INSERT statement. SELECT * displayed all rows and columns as expected.

10 Viva-Voce Questions

Q1. What is the difference between DDL and DML statements?

Q2. What is the difference between CHAR and VARCHAR?

Q3. Can one INSERT statement add more than one row? How?

Q4. What does the * mean in SELECT * FROM student?

EXPERIMENT NO. 2

1 Experiment Title

Updating and Deleting Rows

2 Aim

To change a value in a row, then delete a row, and check the table after each step.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

The **UPDATE** statement changes values that already exist in a table. The **SET** clause names the column and the new value, and the new value can use the old one, for example quantity = quantity + 10.

The **DELETE** statement removes rows from a table. In both statements the **WHERE** clause chooses the rows that are changed; without WHERE every row of the table is affected.

5 Algorithm / Procedure

1. Create the stock table and insert the three set-up rows.
2. Write an UPDATE with SET and WHERE to change the quantity of item_code 1, then run SELECT * to check it.
3. Write an UPDATE that adds 10 to the quantity of item_code 2, then check the table.
4. Write a DELETE with WHERE item_code = 3, then check the table.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE stock (item_code INT, item_name VARCHAR(30), quantity INT);
INSERT INTO stock (item_code, item_name, quantity)
VALUES (1, 'Notebook', 50),
       (2, 'Pen', 120),
       (3, 'Bag', 0);
```

Task 1: Change the quantity of item_code 1 to 80 SQL

```sql
UPDATE stock SET quantity = 80 WHERE item_code = 1;
SELECT * FROM stock;
```

Task 2: Add 10 to the quantity of item_code 2 SQL

```sql
UPDATE stock SET quantity = quantity + 10 WHERE item_code = 2;
SELECT * FROM stock;
```

Task 3: Delete the row of item_code 3 SQL

```sql
DELETE FROM stock WHERE item_code = 3;
SELECT * FROM stock;
```

7 Input

Table structure and test data used in this experiment.

Table stock: structure

Column	Data type	Constraint
item_code	INT	-
item_name	VARCHAR(30)	-
quantity	INT	-

Table stock: data

item_code	item_name	quantity
1	Notebook	50
2	Pen	120
3	Bag	0

8 Output

Output of Task 1: Change the quantity of item_code 1 to 80

```
Query OK, 1 row affected
```

item_code	item_name	quantity
1	Notebook	80
2	Pen	120
3	Bag	0

```
3 rows in set
```

Output of Task 2: Add 10 to the quantity of item_code 2

```
Query OK, 1 row affected
```

item_code	item_name	quantity
1	Notebook	80
2	Pen	130
3	Bag	0

```
3 rows in set
```

Output of Task 3: Delete the row of item_code 3

```
Query OK, 1 row affected
```

item_code	item_name	quantity
1	Notebook	80
2	Pen	130

```
2 rows in set
```

9 Result

The quantity of item 1 was changed to 80, 10 was added to the quantity of item 2, and the row of item 3 was deleted. The table was checked after every step and matched the samples.

10 Viva-Voce Questions

Q1. What happens if UPDATE or DELETE is run without a WHERE clause?

Q2. What is the difference between DELETE, TRUNCATE and DROP?

Q3. How does SET quantity = quantity + 10 work?

Q4. Is DELETE a DDL or a DML command?

EXPERIMENT NO. 3

1 Experiment Title

Selecting Columns and Rows with WHERE

2 Aim

To show some columns of a table, and to show only the rows that match a condition.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

Choosing some columns of a table is called **projection**: the column names are written after SELECT. Choosing some rows is called **selection**: the **WHERE** clause keeps only the rows for which the condition is true.

WHERE can use the comparison operators =, <>, >, <, >= and <=. Text values are written inside single quotes and numbers are written without quotes.

5 Algorithm / Procedure

1. Create the student table and insert the four set-up rows.
2. Write SELECT name, marks FROM student to show only two columns.
3. Add WHERE marks > 60 to show only the matching rows.
4. Add WHERE roll_no = 103 and select only the name column.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
INSERT INTO student (roll_no, name, marks)
VALUES (101, 'Asha', 78),
       (102, 'Rohan', 64),
       (103, 'Neha', 88),
       (104, 'Deepak', 52);
```

Task 1: Show only the name and marks of all students SQL

```sql
SELECT name, marks FROM student;
```

Task 2: Show all columns of students with marks greater than 60 SQL

```sql
SELECT * FROM student
WHERE marks > 60;
```

Task 3: Show the name of the student whose roll_no is 103 SQL

```sql
SELECT name FROM student
WHERE roll_no = 103;
```

7 Input

Table structure and test data used in this experiment.

Table student: structure

Column	Data type	Constraint
roll_no	INT	-
name	VARCHAR(30)	-
marks	INT	-

Table student: data

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88
104	Deepak	52

8 Output

Output of Task 1: Show only the name and marks of all students

name	marks
Asha	78
Rohan	64
Neha	88
Deepak	52

4 rows in set

Output of Task 2: Show all columns of students with marks greater than 60

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88

3 rows in set

Output of Task 3: Show the name of the student whose roll_no is 103

name
Neha

1 row in set

9 Result

The required columns and rows were displayed using SELECT with a column list and with the WHERE clause. All three outputs matched the samples.

10 Viva-Voce Questions

Q1. What is the difference between projection and selection?

Q2. Which clause filters rows in a SELECT statement?

Q3. How are text values written in a condition?

Q4. Name the comparison operators used in WHERE.

EXPERIMENT NO. 4

1 Experiment Title

Combining Conditions

2 Aim

To combine conditions with AND, check a range with BETWEEN, and check a list of values with IN.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

AND joins two conditions and a row is shown only when both are true. **OR** shows the row when at least one is true.

BETWEEN a AND b checks that a value lies in a range and includes both end values. **IN (list)** checks that a value is equal to one of the values in the list; it is a short form of several OR conditions.

5 Algorithm / Procedure

1. Create the player table and insert the four set-up rows.
2. Write a WHERE clause with team = 'Eagles' AND runs > 35.
3. Write a WHERE clause with runs BETWEEN 40 AND 80.
4. Write a WHERE clause with team IN ('Tigers', 'Lions').
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE player (name VARCHAR(30), team VARCHAR(20), runs INT);
INSERT INTO player (name, team, runs)
VALUES ('Asha', 'Eagles', 45),
       ('Meera', 'Tigers', 72),
       ('Arjun', 'Eagles', 30),
       ('Kiran', 'Lions', 88);
```

Task 1: Eagles players with more than 35 runs (AND) SQL

```sql
SELECT name, runs FROM player
WHERE team = 'Eagles' AND runs > 35;
```

Task 2: Players with runs between 40 and 80 (BETWEEN) SQL

```sql
SELECT name, runs FROM player
WHERE runs BETWEEN 40 AND 80;
```

Task 3: Players of Tigers or Lions (IN) SQL

```sql
SELECT name, team FROM player
WHERE team IN ('Tigers', 'Lions');
```

7 Input

Table structure and test data used in this experiment.

Table player: structure

Column	Data type	Constraint
name	VARCHAR(30)	-
team	VARCHAR(20)	-
runs	INT	-

Table player: data

name	team	runs
Asha	Eagles	45
Meera	Tigers	72
Arjun	Eagles	30
Kiran	Lions	88

8 Output

Output of Task 1: Eagles players with more than 35 runs (AND)

name	runs
Asha	45

1 row in set

Output of Task 2: Players with runs between 40 and 80 (BETWEEN)

name	runs
Asha	45
Meera	72

2 rows in set

Output of Task 3: Players of Tigers or Lions (IN)

name	team
Meera	Tigers
Kiran	Lions

2 rows in set

9 Result

Conditions were combined with AND, a range was checked with BETWEEN and a list was checked with IN. The outputs matched the samples.

10 Viva-Voce Questions

Q1. What is the difference between AND and OR?

Q2. Does BETWEEN include the two end values?

Q3. Which operator is a short form of several OR conditions?

Q4. Write a condition to find runs that are not between 40 and 80.

EXPERIMENT NO. 5

1 Experiment Title

Pattern Matching with LIKE

2 Aim

To find titles that match a pattern with the LIKE operator.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

The **LIKE** operator compares a text value with a pattern. Two wildcard characters are used in the pattern: **%** stands for any number of characters (including none) and _ stands for exactly one character.

'Data%' matches text that starts with Data, '%C' matches text that ends with C, and '_h%' matches text whose second letter is h. **NOT LIKE** gives the opposite result.

5 Algorithm / Procedure

1. Create the book table and insert the four set-up rows.
2. Write WHERE title LIKE 'Data%' for titles that start with Data.
3. Write WHERE title LIKE '%C' for titles that end with C.
4. Write WHERE title LIKE '_h%' for titles whose second letter is h.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE book (title VARCHAR(40), category VARCHAR(20), copies INT);
INSERT INTO book (title, category, copies)
VALUES ('Database Systems', 'Computer', 3),
       ('Digital Electronics', 'Electronics', 2),
       ('Data Structures in C', 'Computer', 2),
       ('Short Stories', 'Literature', 1);
```

Task 1: Titles that start with Data SQL

```sql
SELECT title FROM book
WHERE title LIKE 'Data%';
```

Task 2: Titles that end with the letter C SQL

```sql
SELECT title FROM book
WHERE title LIKE '%C';
```

Task 3: Titles whose second letter is h SQL

```sql
SELECT title FROM book
WHERE title LIKE '_h%';
```

7 Input

Table structure and test data used in this experiment.

Table book: structure

Column	Data type	Constraint
title	VARCHAR(40)	-
category	VARCHAR(20)	-
copies	INT	-

Table book: data

title	category	copies
Database Systems	Computer	3
Digital Electronics	Electronics	2
Data Structures in C	Computer	2
Short Stories	Literature	1

8 Output

Output of Task 1: Titles that start with Data

title
Database Systems
Data Structures in C

2 rows in set

Output of Task 2: Titles that end with the letter C

title
Data Structures in C

1 row in set

Output of Task 3: Titles whose second letter is h

title
Short Stories

1 row in set

9 Result

Titles were found by pattern using LIKE with the % and _ wildcards. All three outputs matched the samples.

10 Viva-Voce Questions

Q1. What do the % and _ wildcards mean?

Q2. Which pattern finds titles that contain the word Data anywhere?

Q3. What does NOT LIKE do?

Q4. Is LIKE case sensitive in MySQL by default?

EXPERIMENT NO. 6

1 Experiment Title

Sorting Rows and Removing Repeats

2 Aim

To sort the rows of a result, remove repeated values, and show only some rows.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

ORDER BY sorts the result. **ASC** (ascending) is the default and **DESC** sorts from the highest to the lowest value.

DISTINCT removes repeated values from the result. **LIMIT n** shows only the first n rows, so it is used after ORDER BY to get the top rows.

5 Algorithm / Procedure

1. Create the product table and insert the four set-up rows.
2. Write SELECT with ORDER BY price DESC.
3. Write SELECT DISTINCT category with ORDER BY category.
4. Write ORDER BY price DESC with LIMIT 2 to get the two most expensive products.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE product (product_name VARCHAR(30), category VARCHAR(20), price INT);
INSERT INTO product (product_name, category, price)
VALUES ('Pen', 'Stationery', 15),
       ('Notebook', 'Stationery', 60),
       ('Calculator', 'Electronics', 850),
       ('Bag', 'Office', 500);
```

Task 1: Products sorted by price, highest first SQL

```sql
SELECT product_name, price FROM product
ORDER BY price DESC;
```

Task 2: Each category once, sorted by category SQL

```sql
SELECT DISTINCT category FROM product
ORDER BY category;
```

Task 3: The two most expensive products SQL

```sql
SELECT product_name, price FROM product
ORDER BY price DESC
LIMIT 2;
```

7 Input

Table structure and test data used in this experiment.

Table product: structure

Column	Data type	Constraint
product_name	VARCHAR(30)	-
category	VARCHAR(20)	-
price	INT	-

Table product: data

product_name	category	price
Pen	Stationery	15
Notebook	Stationery	60
Calculator	Electronics	850
Bag	Office	500

8 Output

Output of Task 1: Products sorted by price, highest first

product_name	price
Calculator	850
Bag	500
Notebook	60
Pen	15

4 rows in set

Output of Task 2: Each category once, sorted by category

category
Electronics
Office
Stationery

3 rows in set

Output of Task 3: The two most expensive products

product_name	price
Calculator	850
Bag	500

2 rows in set

9 Result

Rows were sorted with ORDER BY, repeated categories were removed with DISTINCT and the top two rows were selected with LIMIT. The outputs matched the samples.

10 Viva-Voce Questions

Q1. What is the default sorting order of ORDER BY?

Q2. What does DISTINCT do?

Q3. Why is LIMIT used together with ORDER BY?

Q4. Can ORDER BY use more than one column?

EXPERIMENT NO. 7

1 Experiment Title

Using Aggregate Functions

2 Aim

To use one aggregate function in each query: COUNT, SUM and MAX.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

An **aggregate function** works on many rows and returns a single value. **COUNT(*)** counts rows, **SUM(col)** adds the values, **MAX(col)** and **MIN(col)** give the highest and lowest value, and **AVG(col)** gives the average.

A column alias written with **AS** gives the result column a readable name such as num_sales or total.

5 Algorithm / Procedure

1. Create the sales table and insert the four set-up rows.
2. Write SELECT COUNT(*) AS num_sales to count the rows.
3. Write SELECT SUM(amount) AS total to add all the amounts.
4. Write SELECT MAX(amount) AS highest to find the highest amount.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE sales (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO sales (salesperson, region, amount)
VALUES ('Deepak', 'North', 12000),
       ('Pooja', 'South', 8500),
       ('Deepak', 'South', 9500),
       ('Lata', 'North', 11000);
```

Task 1: Count the sales rows SQL

```sql
SELECT COUNT(*) AS num_sales FROM sales;
```

Task 2: Total of all amounts SQL

```sql
SELECT SUM(amount) AS total FROM sales;
```

Task 3: Highest amount SQL

```sql
SELECT MAX(amount) AS highest FROM sales;
```

7 Input

Table structure and test data used in this experiment.

Table sales: structure

Column	Data type	Constraint
salesperson	VARCHAR(20)	-
region	VARCHAR(20)	-
amount	INT	-

Table sales: data

salesperson	region	amount
Deepak	North	12000
Pooja	South	8500
Deepak	South	9500
Lata	North	11000

8 Output

Output of Task 1: Count the sales rows

num_sales
4

1 row in set

Output of Task 2: Total of all amounts

total
41000

1 row in set

Output of Task 3: Highest amount

highest
12000

1 row in set

9 Result

The aggregate functions COUNT, SUM and MAX were used and returned 4 rows, a total of 41000 and a highest amount of 12000.

10 Viva-Voce Questions

Q1. Name five aggregate functions.

Q2. What is the difference between COUNT(*) and COUNT(column)?

Q3. Can an aggregate function be used in a WHERE clause?

Q4. Why do we use AS with an aggregate function?

EXPERIMENT NO. 8

1 Experiment Title

Grouping Rows and Using HAVING

2 Aim

To group rows by region and to check the totals of each group.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

GROUP BY puts rows that have the same value in a column into one group, and an aggregate function then gives one result for each group.

WHERE filters rows before grouping, but **HAVING** filters the groups after the aggregate values are calculated. So a condition on SUM or COUNT is written in HAVING.

5 Algorithm / Procedure

1. Create the shop_sale table and insert the four set-up rows.
2. Write SELECT region, COUNT(*) with GROUP BY region and ORDER BY region.
3. Write SELECT region, SUM(amount) with GROUP BY region.
4. Add HAVING SUM(amount) > 20000 to keep only the large groups.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE shop_sale (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO shop_sale (salesperson, region, amount)
VALUES ('Deepak', 'North', 12000),
       ('Pooja', 'South', 8500),
       ('Deepak', 'South', 9500),
       ('Lata', 'North', 11000);
```

Task 1: Number of sales in each region SQL

```sql
SELECT region, COUNT(*) AS num_sales
FROM shop_sale
GROUP BY region
ORDER BY region;
```

Task 2: Total amount in each region SQL

```sql
SELECT region, SUM(amount) AS total
FROM shop_sale
GROUP BY region
ORDER BY region;
```

Task 3: Regions whose total is more than 20000 SQL

```sql
SELECT region, SUM(amount) AS total
FROM shop_sale
GROUP BY region
HAVING SUM(amount) > 20000
ORDER BY region;
```

7 Input

Table structure and test data used in this experiment.

Table shop_sale: structure

Column	Data type	Constraint
salesperson	VARCHAR(20)	-
region	VARCHAR(20)	-
amount	INT	-

Table shop_sale: data

salesperson	region	amount
Deepak	North	12000
Pooja	South	8500
Deepak	South	9500
Lata	North	11000

8 Output

Output of Task 1: Number of sales in each region

region	num_sales
North	2
South	2

2 rows in set

Output of Task 2: Total amount in each region

region	total
North	23000
South	18000

2 rows in set

Output of Task 3: Regions whose total is more than 20000

region	total
North	23000

1 row in set

9 Result

Rows were grouped by region, the count and total of each group were found, and HAVING kept only the region with a total above 20000.

10 Viva-Voce Questions

Q1. What does GROUP BY do?

Q2. What is the difference between WHERE and HAVING?

Q3. Can we use HAVING without GROUP BY?

Q4. Why must every non-aggregate column in SELECT appear in GROUP BY?

EXPERIMENT NO. 9

1 Experiment Title

Aliases and CASE

2 Aim

To give columns new names and to show a value chosen by a condition.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

An **alias** gives a column (or table) a temporary name with the keyword **AS**. It changes only the heading in the result, not the table.

CASE WHEN condition THEN value ... ELSE value END works like if-else inside a query: the first condition that is true decides the value shown. A calculation such as marks + 5 can also be shown as a new column.

5 Algorithm / Procedure

1. Create the exam_result table and insert the four set-up rows.
2. Write SELECT name AS student_name, marks AS score.
3. Write a CASE expression that gives Pass for marks of 40 or more, otherwise Fail.
4. Write marks + 5 AS with_bonus.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE exam_result (name VARCHAR(20), marks INT);
INSERT INTO exam_result (name, marks)
VALUES ('Asha', 92),
       ('Rohan', 47),
       ('Neha', 68),
       ('Meera', 33);
```

Task 1: Rename the columns with AS SQL

```sql
SELECT name AS student_name, marks AS score
FROM exam_result;
```

Task 2: Pass or Fail with CASE SQL

```sql
SELECT name,
       CASE WHEN marks >= 40 THEN 'Pass'
            ELSE 'Fail'
       END AS result
FROM exam_result;
```

Task 3: Marks with 5 bonus marks SQL

```sql
SELECT name, marks + 5 AS with_bonus
FROM exam_result;
```

7 Input

Table structure and test data used in this experiment.

Table exam_result: structure

Column	Data type	Constraint
name	VARCHAR(20)	-
marks	INT	-

Table exam_result: data

name	marks
Asha	92
Rohan	47
Neha	68
Meera	33

8 Output

Output of Task 1: Rename the columns with AS

student_name	score
Asha	92
Rohan	47
Neha	68
Meera	33

4 rows in set

Output of Task 2: Pass or Fail with CASE

name	result
Asha	Pass
Rohan	Pass
Neha	Pass
Meera	Fail

4 rows in set

Output of Task 3: Marks with 5 bonus marks

name	with_bonus
Asha	97
Rohan	52
Neha	73
Meera	38

4 rows in set

9 Result

Column aliases were created with AS, Pass/Fail was shown with CASE, and the bonus marks were calculated in a new column. The outputs matched the samples.

10 Viva-Voce Questions

Q1. What is an alias and why is it used?

Q2. Does an alias change the column name in the table?

Q3. How does the CASE expression work?

Q4. What does the ELSE part of CASE do?

EXPERIMENT NO. 10

1 Experiment Title

Joining Two Tables with INNER JOIN

2 Aim

To join a department table and an employee table on dept_id.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

A **JOIN** combines rows of two tables using a related column. **INNER JOIN** returns only the rows that have a matching value in both tables; here dept_id links an employee to a department.

The condition is written after **ON**. Table aliases such as e and d make the query shorter. Department 40 (Research) has no employees, so it does not appear in an inner join.

5 Algorithm / Procedure

1. Create the department and employee tables and insert the set-up rows.
2. Write SELECT with INNER JOIN ... ON e.dept_id = d.dept_id and ORDER BY emp_name.
3. Add WHERE d.dept_name = 'Sales' to show only the Sales employees.
4. Show the salary as well and use ORDER BY salary DESC.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE department (dept_id INT, dept_name VARCHAR(20));
INSERT INTO department (dept_id, dept_name)
VALUES (10, 'Accounts'),
       (20, 'Sales'),
       (40, 'Research');

CREATE TABLE employee (emp_name VARCHAR(30), dept_id INT, salary INT);
INSERT INTO employee (emp_name, dept_id, salary)
VALUES ('Anita', 10, 90000),
       ('Lata', 20, 55000),
       ('Deepak', 10, 60000),
       ('Pooja', 20, 42000);
```

Task 1: Employee name with department name SQL

```sql
SELECT e.emp_name, d.dept_name
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id
ORDER BY e.emp_name;
```

Task 2: Employees of the Sales department SQL

```sql
SELECT e.emp_name
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Sales'
ORDER BY e.emp_name;
```

Task 3: Name, department and salary (highest salary first) SQL

```sql
SELECT e.emp_name, d.dept_name, e.salary
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id
ORDER BY e.salary DESC;
```

7 Input

Table structure and test data used in this experiment.

Table department: structure

Column	Data type	Constraint
dept_id	INT	-
dept_name	VARCHAR(20)	-

Table department: data

dept_id	dept_name
10	Accounts
20	Sales
40	Research

Table employee: structure

Column	Data type	Constraint
emp_name	VARCHAR(30)	-
dept_id	INT	-
salary	INT	-

Table employee: data

emp_name	dept_id	salary
Anita	10	90000
Lata	20	55000
Deepak	10	60000
Pooja	20	42000

8 Output

Output of Task 1: Employee name with department name

emp_name	dept_name
Anita	Accounts
Deepak	Accounts
Lata	Sales
Pooja	Sales

4 rows in set

Output of Task 2: Employees of the Sales department

emp_name
Lata
Pooja

2 rows in set

Output of Task 3: Name, department and salary (highest salary first)

emp_name	dept_name	salary
Anita	Accounts	90000
Deepak	Accounts	60000
Lata	Sales	55000
Pooja	Sales	42000

4 rows in set

9 Result

The employee and department tables were joined with INNER JOIN on dept_id, and the outputs matched the samples.

10 Viva-Voce Questions

Q1. What does INNER JOIN return?

Q2. Why is the Research department not in the result?

Q3. What is the use of the ON clause?

Q4. What is a table alias?

EXPERIMENT NO. 11

1 Experiment Title

Other Kinds of Joins

2 Aim

To use a left join, a cross join and a natural join on two small tables.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

LEFT JOIN returns every row of the left table; if there is no match in the right table, the right-side columns are shown as NULL (the Lions team has no players).

CROSS JOIN pairs every row of one table with every row of the other (3 teams x 3 players = 9 rows).
NATURAL JOIN joins the tables automatically on the columns that have the same name, here team_id.

5 Algorithm / Procedure

1. Create the team and player tables and insert the set-up rows.
2. Write team LEFT JOIN player ON team_id and sort by team name, then player name.
3. Write team CROSS JOIN player and sort the same way.
4. Write player NATURAL JOIN team and sort by player name.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE team (team_id INT, team_name VARCHAR(20));
INSERT INTO team (team_id, team_name)
VALUES (1, 'Eagles'),
       (2, 'Tigers'),
       (3, 'Lions');

CREATE TABLE player (player_name VARCHAR(30), team_id INT, age INT);
INSERT INTO player (player_name, team_id, age)
VALUES ('Asha', 1, 24),
       ('Meera', 1, 22),
       ('Arjun', 2, 27);
```

Task 1: LEFT JOIN: every team with its players SQL

```sql
SELECT t.team_name, p.player_name
FROM team t
LEFT JOIN player p ON t.team_id = p.team_id
ORDER BY t.team_name, p.player_name;
```

Task 2: CROSS JOIN: every team and player pair SQL

```sql
SELECT t.team_name, p.player_name
FROM team t
CROSS JOIN player p
ORDER BY t.team_name, p.player_name;
```

Task 3: NATURAL JOIN: player with team name SQL

```sql
SELECT player_name, team_name
FROM player
NATURAL JOIN team
ORDER BY player_name;
```

7 Input

Table structure and test data used in this experiment.

Table team: structure

Column	Data type	Constraint
team_id	INT	-
team_name	VARCHAR(20)	-

Table team: data

team_id	team_name
1	Eagles
2	Tigers
3	Lions

Table player: structure

Column	Data type	Constraint
player_name	VARCHAR(30)	-
team_id	INT	-
age	INT	-

Table player: data

player_name	team_id	age
Asha	1	24
Meera	1	22
Arjun	2	27

8 Output

Output of Task 1: LEFT JOIN: every team with its players

team_name	player_name
Eagles	Asha
Eagles	Meera
Lions	NULL
Tigers	Arjun

4 rows in set

Output of Task 2: CROSS JOIN: every team and player pair

team_name	player_name
Eagles	Arjun
Eagles	Asha
Eagles	Meera
Lions	Arjun
Lions	Asha
Lions	Meera
Tigers	Arjun
Tigers	Asha
Tigers	Meera

9 rows in set

Output of Task 3: NATURAL JOIN: player with team name

player_name	team_name
Arjun	Tigers
Asha	Eagles
Meera	Eagles

3 rows in set

9 Result

LEFT JOIN, CROSS JOIN and NATURAL JOIN were performed. The left join showed the team without players, the cross join gave 9 pairs and the natural join matched on team_id.

10 Viva-Voce Questions

Q1. What is the difference between INNER JOIN and LEFT JOIN?

Q2. How many rows does a CROSS JOIN of 3 rows and 3 rows give?

Q3. On which columns does NATURAL JOIN work?

Q4. What is shown for a missing match in a LEFT JOIN?

EXPERIMENT NO. 12

1 Experiment Title

Using a Subquery

2 Aim

To use one query inside another query with IN.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

A **subquery** is a SELECT statement written inside another statement, in brackets. The inner query runs first and its result is used by the outer query.

With **IN**, the outer query keeps the rows whose value appears in the list returned by the subquery. Here it finds the names that are in both clubs.

5 Algorithm / Procedure

1. Create the club_a and club_b tables and insert the set-up rows.
2. Write SELECT ... FROM club_a WHERE student_name IN (SELECT student_name FROM club_b).
3. Write SELECT COUNT(*) AS members FROM club_b.
4. Write SELECT from club_a with ORDER BY student_name.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name)
VALUES ('Asha'), ('Rohan'), ('Neha');

CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name)
VALUES ('Neha'), ('Pooja'), ('Meera');
```

Task 1: Names in club_a that are also in club_b SQL

```sql
SELECT student_name FROM club_a
WHERE student_name IN (SELECT student_name FROM club_b);
```

Task 2: Number of names in club_b SQL

```sql
SELECT COUNT(*) AS members FROM club_b;
```

Task 3: Names in club_a, sorted SQL

```sql
SELECT student_name FROM club_a
ORDER BY student_name;
```

7 Input

Table structure and test data used in this experiment.

Table club_a: structure

Column	Data type	Constraint
student_name	VARCHAR(30)	-

Table club_a: data

student_name
Asha
Rohan
Neha

Table club_b: structure

Column	Data type	Constraint
student_name	VARCHAR(30)	-

Table club_b: data

student_name
Neha
Pooja
Meera

8 Output

Output of Task 1: Names in club_a that are also in club_b

student_name
Neha

1 row in set

Output of Task 2: Number of names in club_b

members
3

1 row in set

Output of Task 3: Names in club_a, sorted

student_name
Asha
Neha
Rohan

3 rows in set

9 Result

A subquery with IN found the name common to both clubs, and the count and sorted list were displayed correctly.

10 Viva-Voce Questions

Q1. What is a subquery?

Q2. Which query runs first, the inner or the outer?

Q3. What does the IN operator do with a subquery?

Q4. What is the difference between a subquery and a join?

EXPERIMENT NO. 13

1 Experiment Title

UNION and INTERSECT

2 Aim

To combine two lists of names, and to find the names that are in both lists.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

UNION combines the results of two SELECT statements and removes repeated rows. **UNION ALL** keeps the repeated rows. **INTERSECT** returns only the rows that appear in both results.

Both SELECT statements must have the same number of columns with compatible data types. ORDER BY is written once, at the end of the whole statement.

5 Algorithm / Procedure

1. Create the club_a and club_b tables and insert the set-up rows.
2. Write the two SELECT statements joined with UNION and add ORDER BY at the end.
3. Write the two SELECT statements joined with INTERSECT.
4. Write the two SELECT statements joined with UNION ALL.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name)
VALUES ('Asha'), ('Rohan'), ('Neha');

CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name)
VALUES ('Neha'), ('Pooja'), ('Meera');
```

Task 1: UNION: all names, each once SQL

```sql
SELECT student_name FROM club_a
UNION
SELECT student_name FROM club_b
ORDER BY student_name;
```

Task 2: INTERSECT: names in both clubs SQL

```sql
SELECT student_name FROM club_a
INTERSECT
SELECT student_name FROM club_b
ORDER BY student_name;
```

Task 3: UNION ALL: all names with repeats SQL

```sql
SELECT student_name FROM club_a
UNION ALL
SELECT student_name FROM club_b
ORDER BY student_name;
```

7 Input

Table structure and test data used in this experiment.

Table club_a: structure

Column	Data type	Constraint
student_name	VARCHAR(30)	-

Table club_a: data

student_name
Asha
Rohan
Neha

Table club_b: structure

Column	Data type	Constraint
student_name	VARCHAR(30)	-

Table club_b: data

student_name
Neha
Pooja
Meera

8 Output

Output of Task 1: UNION: all names, each once

student_name
Asha
Meera
Neha
Pooja
Rohan

5 rows in set

Output of Task 2: INTERSECT: names in both clubs

student_name
Neha

1 row in set

Output of Task 3: UNION ALL: all names with repeats

student_name
Asha
Meera
Neha
Neha
Pooja
Rohan

6 rows in set

9 Result

UNION gave 5 names, INTERSECT gave the one common name (Neha) and UNION ALL gave 6 names including the repeat. The outputs matched the samples.

10 Viva-Voce Questions

Q1. What is the difference between UNION and UNION ALL?

Q2. What does INTERSECT return?

Q3. What rules must the two SELECT statements follow?

Q4. Where is ORDER BY written in a UNION query?

EXPERIMENT NO. 14

1 Experiment Title

Creating and Using a View

2 Aim

To save a query as a view and then read the view.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

A **view** is a saved SELECT statement that behaves like a virtual table. It stores the query, not the data, so it always shows the current data of the base table.

Views make long queries easy to reuse and can hide columns or rows that a user should not see. A view is created with **CREATE VIEW name AS SELECT ...** and is read with SELECT like any table.

5 Algorithm / Procedure

1. Create the score table and insert the four set-up rows.
2. Write CREATE VIEW high_scores AS SELECT name, marks ... WHERE marks >= 70.
3. Run SELECT * FROM high_scores.
4. Run SELECT name FROM high_scores ORDER BY name.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE score (name VARCHAR(30), subject VARCHAR(20), marks INT);
INSERT INTO score (name, subject, marks)
VALUES ('Asha', 'Maths', 78),
       ('Rohan', 'Maths', 64),
       ('Neha', 'Maths', 88),
       ('Kiran', 'Maths', 52);
```

Task 1: Create the view high_scores SQL

```sql
CREATE VIEW high_scores AS
SELECT name, marks FROM score
WHERE marks >= 70;
```

Task 2: Show all rows of the view SQL

```sql
SELECT * FROM high_scores;
```

Task 3: Show only the names, sorted SQL

```sql
SELECT name FROM high_scores
ORDER BY name;
```

7 Input

Table structure and test data used in this experiment.

Table score: structure

Column	Data type	Constraint
name	VARCHAR(30)	-
subject	VARCHAR(20)	-
marks	INT	-

Table score: data

name	subject	marks
Asha	Maths	78
Rohan	Maths	64
Neha	Maths	88
Kiran	Maths	52

8 Output

Output of Task 1: Create the view high_scores

```
Query OK, view created
```

Output of Task 2: Show all rows of the view

name	marks
Asha	78
Neha	88

2 rows in set

Output of Task 3: Show only the names, sorted

name
Asha
Neha

2 rows in set

9 Result

The view high_scores was created and read like a table. It showed only the students with 70 or more marks.

10 Viva-Voce Questions

Q1. What is a view?

Q2. Does a view store data?

Q3. Give two advantages of using a view.

Q4. How do you remove a view?

EXPERIMENT NO. 15

1 Experiment Title

Indexes and Adding a Row

2 Aim

To create an index on a column, read rows by that column, and add a row.

3 Software / Tools Required

MySQL 8.x database server with MySQL Workbench (or the mysql command-line client). Any standard SQL database can be used.

4 Theory

An **index** is a structure that helps the database find rows quickly, like the index of a book. It is created with **CREATE INDEX name ON table (column)** and is useful on columns that are often used in WHERE.

An index makes reading faster but takes extra space and makes INSERT and UPDATE a little slower. The PRIMARY KEY column customer_id already has an index and cannot contain duplicate or NULL values.

5 Algorithm / Procedure

1. Create the customer table with PRIMARY KEY and NOT NULL, and insert the four set-up rows.
2. Write CREATE INDEX idx_city ON customer (city).
3. Select the names of the customers who live in Delhi, sorted by name.
4. Add the customer 5, Arjun, Pune with one INSERT and display the table.
5. Compare each result with the sample output.

6 Program / Queries / Commands

Set-up: create the table(s) and insert the data (not a task) SQL

```sql
CREATE TABLE customer (
    customer_id  INT PRIMARY KEY,
    name         VARCHAR(30) NOT NULL,
    city         VARCHAR(20)
);
INSERT INTO customer (customer_id, name, city)
VALUES (1, 'Kiran', 'Kochi'),
       (2, 'Deepak', 'Delhi'),
       (3, 'Lata', 'Mumbai'),
       (4, 'Asha', 'Delhi');
```

Task 1: Create the index idx_city SQL

```sql
CREATE INDEX idx_city ON customer (city);
```

Task 2: Customers who live in Delhi SQL

```sql
SELECT name FROM customer
WHERE city = 'Delhi'
ORDER BY name;
```

Task 3: Add customer 5 and show the table SQL

```sql
INSERT INTO customer (customer_id, name, city)
VALUES (5, 'Arjun', 'Pune');
SELECT * FROM customer ORDER BY customer_id;
```

7 Input

Table structure and test data used in this experiment.

Table customer: structure

Column	Data type	Constraint
customer_id	INT	PRIMARY KEY
name	VARCHAR(30)	NOT NULL
city	VARCHAR(20)	-

Table customer: data

customer_id	name	city
1	Kiran	Kochi
2	Deepak	Delhi
3	Lata	Mumbai
4	Asha	Delhi

8 Output

Output of Task 1: Create the index idx_city

```
Query OK, index created
```

Output of Task 2: Customers who live in Delhi

name
Asha
Deepak

```
2 rows in set
```

Output of Task 3: Add customer 5 and show the table

```
Query OK, 1 row affected
```

customer_id	name	city
1	Kiran	Kochi
2	Deepak	Delhi
3	Lata	Mumbai
4	Asha	Delhi
5	Arjun	Pune

```
5 rows in set
```

9 Result

The index idx_city was created on the city column, the Delhi customers were selected, and a new customer was added. The table matched the sample.

10 Viva-Voce Questions

Q1. What is an index and why is it used?

Q2. What is the disadvantage of having too many indexes?

Q3. What is the difference between PRIMARY KEY and a normal index?

Q4. What happens if you insert a duplicate customer_id?